

02 — Instruction-Level Parallelism e Execução Superescalar

Pré-requisito: [01-cpu-execution-pipeline.md](#)

1. Intuição: e se, além de sobrepor, a CPU também duplicasse os circuitos?

O arquivo anterior mostrou que pipelining faz uma instrução **terminar** por ciclo, no regime estável, sobrepondo etapas. Mas por que parar em uma? Se o chip tem espaço (e transistores) suficientes, por que não ter **duas** unidades de EXECUTE, capazes de rodar duas instruções ao mesmo tempo, no mesmo ciclo?

Isso é exatamente o que **CPUs superescalares** fazem — praticamente toda CPU moderna (desde os anos 90) é superescalar. E a capacidade de explorar isso — encontrar instruções independentes o suficiente pra rodar juntas — se chama **Instruction-Level Parallelism (ILP)**.

2. O que "superescalar" significa concretamente

CPU escalar (pipeline simples, 1 instrução por estágio):

Ciclo:	1	2	3	4
Instr 1:	FETCH	DECODE	EXEC	WB
Instr 2:		FETCH	DECODE	EXEC WB

CPU superescalar (duas instruções por estágio, em paralelo):

Ciclo:	1	2	3	4
Instr 1:	FETCH	DECODE	EXEC	WB
Instr 2:	FETCH	DECODE	EXEC	WB
				<- rodando JUNTO com Instr 1, no MESMO ciclo
Instr 3:		FETCH	DECODE	EXEC
Instr 4:		FETCH	DECODE	EXEC

O chip literalmente tem múltiplas unidades de execução (múltiplas ULAs, por exemplo) que podem operar simultaneamente. Um processador moderno típico consegue despachar 3-6 instruções por ciclo, se elas forem independentes o suficiente.

3. ILP: a métrica de "quanto paralelismo existe pra explorar"

Instruction-Level Parallelism é a medida de quantas instruções de um trecho de código **poderiam**, em teoria, rodar ao mesmo tempo, dado que não têm dependência entre si.

```
int a = x + y;    // instrução 1
int b = p + q;    // instrução 2 -- independente da 1!
int c = a + b;    // instrução 3 -- depende de 1 E 2
```

Instruções 1 e 2 têm ILP entre si (nenhuma usa o resultado da outra) — uma CPU superescalar pode rodar as duas no mesmo ciclo. Instrução 3 **não** pode rodar junto com nenhuma das duas — ela precisa dos resultados de ambas primeiro.

PERGUNTA RÁPIDA 1: no trecho abaixo, quais instruções têm ILP entre si (podem, em teoria, rodar juntas), e quais formam uma cadeia de dependência?

```
int s1 = a + b;  
int s2 = c + d;  
int s3 = s1 + s2;  
int s4 = e + f;  
int s5 = s3 + s4;
```

► Resposta

s1, **s2** e **s4** são independentes entre si — os três podem, em teoria, rodar no mesmo ciclo (ILP = 3 nesse ponto). **s3** depende de **s1** e **s2** (precisa esperar os dois). **s5** depende de **s3** e **s4**. A cadeia de dependência mais longa (**s1/s2** → **s3** → **s5**) é o que limita o tempo mínimo possível, mesmo com paralelismo infinito disponível — isso se chama **critical path** (caminho crítico), e é um conceito que reaparece inalterado quando você estudar paralelismo de GPU.

4. Execução fora de ordem (out-of-order execution)

Um problema prático: e se a instrução 2 do seu código estiver esperando um dado vindo da memória (lento — arquivo 04 explica por quê), mas a instrução 3, mais abaixo no código, já está pronta pra rodar e não depende de nada que ainda falta?

CPUs modernas **reordenam** instruções internamente (sem mudar o resultado final, é claro) pra manter as unidades de execução ocupadas: elas despacham a instrução 3 primeiro, antes da 2, se a 3 está pronta e a 2 não está. Isso é feito por um circuito de hardware sofisticado (a "janela de instruções", o "reorder buffer") — você não precisa reordenar seu código manualmente para isso acontecer; a CPU faz isso automaticamente, dentro de uma janela de algumas dezenas/centenas de instruções à frente.

5. Por que você não controla ILP diretamente, mas influencia ele

Você não escreve "rode isso em paralelo" pra ILP (diferente de threads ou SIMD, que são paralelismo explícito). ILP é algo que o **compilador e o hardware** exploram automaticamente, a partir de como você escreve o código. Mas você influencia isso:

- **Cadeias de dependência longas matam ILP.** Uma soma acumulada sequencial (`soma += arr[i]` num loop) cria uma cadeia onde cada iteração depende do resultado da anterior — ILP = 1, nenhuma sobreposição possível entre iterações.
- **Múltiplos acumuladores independentes destravam ILP.** Reescrever a mesma soma usando 4 acumuladores separados, somados no final, dá ao hardware 4 cadeias independentes pra intercalar.

```
// ILP ruim: uma cadeia de dependência só
float soma = 0;
for (int i = 0; i < n; i++) soma += arr[i];

// ILP melhor: quatro cadeias independentes, combinadas no final
float s0 = 0, s1 = 0, s2 = 0, s3 = 0;
for (int i = 0; i < n; i += 4) {
    s0 += arr[i];
    s1 += arr[i+1];
    s2 += arr[i+2];
    s3 += arr[i+3];
}
float soma = s0 + s1 + s2 + s3;
```

PERGUNTA RÁPIDA 2: por que a segunda versão pode dar um resultado **numericamente ligeiramente diferente** da primeira, mesmo somando exatamente os mesmos números?

► Resposta

Ponto flutuante não é associativo: $(a + b) + c$ pode não ser bit-a-bit igual a $a + (b + c)$, por causa de arredondamento em cada soma intermediária. Somar em ordem diferente (quatro cadeias separadas, combinadas no final, em vez de uma soma sequencial) muda a ordem das operações de ponto flutuante, o que pode mudar o resultado no último bit. Isso não é bug — é uma consequência conhecida de otimizar reduções, e é exatamente o tipo de coisa que o seu lab trata como "decisão de correção, não só velocidade" (lembra do `-Ofast/--use_fast_math` do Módulo 07? É o mesmo fenômeno).

6. Por que isso importa para CUDA

O conceito de **critical path** (cadeia de dependência mais longa determinando o tempo mínimo, mesmo com paralelismo disponível) é idêntico em GPU — só que em escala muito maior (milhares de threads em vez de uma pipeline de instrução). Quando você projetar uma redução em GPU (somar um array inteiro usando muitas threads), o mesmo princípio de "múltiplos acumuladores independentes reduzem o caminho crítico" reaparece — só que agora os "acumuladores" são threads/warps diferentes, e a técnica se chama redução em árvore.

7. Erros comuns

- Achar que ILP é algo que você "ativa" com uma flag. Ele é uma propriedade do código (quanta independência existe entre instruções vizinhas) que o hardware explora até um limite físico (quantas unidades de execução existem, quão grande é a janela de reordenação).
- Confundir ILP com paralelismo de thread. ILP acontece **dentro de uma única thread**, automaticamente. Múltiplas threads é paralelismo explícito, outro assunto.
- Assumir que reescrever pra "mais independência" sempre ajuda sem medir — em código simples, o compilador já pode ter feito essa reorganização sozinho (isso se chama auto-vetorização/reordenação do compilador). Vale medir antes de assumir.

PREDICT BEFORE RUNNING (conceitual)

```
int a = x1 * x2;  
int b = a + x3;  
int c = x4 * x5;  
int d = x6 * x7;  
int e = c + d;  
int f = b + e;
```

1. Desenhe (mentalmente ou no papel) o grafo de dependência dessas 6 instruções.
2. Qual é o critical path (a cadeia de dependência mais longa, em número de instruções)?
3. Com unidades de execução infinitas disponíveis, qual o número mínimo de "passos" pra calcular **f**?

► Explicação

a e **c** e **d** não dependem de nada (podem rodar no passo 1). **b** depende de **a**; **e** depende de **c** e **d** — os dois podem rodar no passo 2. **f** depende de **b** e **e** — passo 3. Critical path: **x1, x2** → **a** → **b** → **f** (ou **x4, x5/x6, x7** → **c/d** → **e** → **f**), ambos com 3 passos. Mesmo com execução infinitamente paralela disponível, **f** não pode ser calculado antes do passo 3 — esse é o limite físico imposto pela dependência de dados, não pela quantidade de hardware.